

BANK MANAGEMENT SYSTEM

Python + SQLite Mini Project Documentation

Developed By

Anin Shaj

A console-based banking application supporting user registration, login, bank account management, deposits, withdrawals, money transfers and transaction history.

1. Abstract

The Bank Management System is a Python-based console application developed using Python and SQLite. The system provides basic banking operations through a simple menu-driven interface. Users can register and log in, create a bank account, view account details and balance, deposit and withdraw money, transfer money between accounts, view transaction history, and manage their profile. SQLite is used to persist the application data in a local database named **bankmanagement.db**.

2. Introduction

Banking applications require reliable handling of user information, account information and financial transactions. This mini project demonstrates these concepts using Python programming and a relational SQLite database. The project connects application logic with database operations so that banking data remains available between program executions.

3. Problem Statement

A basic banking application needs a structured way to authenticate users, maintain accounts, update balances and record transactions. Without a database-backed system, maintaining consistent account and transaction information becomes difficult. This project addresses the problem by combining Python functions and SQLite tables into a single working banking system.

4. Objectives

- Implement user registration and login.
- Create and maintain bank accounts.
- Allow balance checking and account-detail viewing.
- Support deposits and withdrawals with transaction records.
- Support transfers between bank accounts.
- Maintain transaction history.
- Use SQLite to store related banking data.
- Apply basic validation, exception handling and password hashing.

5. Technologies Used

Technology	Purpose
Python	Application logic and console interface
SQLite	Local relational database
PyCharm	Development environment
DB Browser for SQLite	Database inspection and verification

6. System Requirements

Software: Python 3.x, PyCharm, and DB Browser for SQLite.

Hardware: A standard computer capable of running Python and PyCharm with sufficient storage for the local SQLite database.

7. System Design

The application follows a menu-driven console architecture. A user first registers and logs in. After successful authentication, the customer dashboard provides account and transaction operations. Database tables are connected through primary-key and foreign-key relationships.

8. Database Design

The project uses three related tables: **registration**, **account**, and **transactions**.

Table	Important fields	Purpose
registration	user_id, username, email, password	Stores user registration and login information
account	account_no, user_id, name, age, account_type, balance	Stores bank account details and balance
transactions	transaction_id, account_no, transaction_type, amount, description, date_time	Stores transaction history

9. ER Diagram

The following relationship model represents the database structure used by the application.

REGISTRATION PK user_id username email password	ACCOUNT PK account_no FK user_id name age account_type balance	TRANSACTIONS PK transaction_id FK account_no transaction_type amount description date_time
--	---	---

Relationship: one registered user can be associated with an account through user_id; an account can have multiple transaction records through account_no.

10. Modules

- **Registration Module:** Creates a new user and validates unique username/email.
- **Login Module:** Authenticates a registered user.
- **Account Module:** Creates an account and displays account details and balance.
- **Deposit Module:** Adds money to the account and records the transaction.
- **Withdrawal Module:** Removes money after balance validation and records the transaction.
- **Transfer Module:** Transfers money between accounts and records transaction entries.

- **Transaction History Module:** Displays stored transaction records.
- **Profile Module:** Displays and updates user profile information.

11. Key Features

- Menu-driven console interface.
- SQLite database persistence.
- Foreign-key support for related tables.
- Password hashing using SHA-256.
- Basic email/input validation.
- Deposit and withdrawal balance updates.
- Account-to-account transfer functionality.
- Transaction history with date/time information.
- Exception handling and rollback for transaction failures.

12. Implementation

The main program is contained in **miniproject.py**. The database connection uses **bankmanagement.db**. SQLite's foreign-key support is enabled, and SQL statements are used to create tables, insert records, update balances and retrieve information.

The application uses the Python standard library, including sqlite3, hashlib, re and datetime. No third-party Python package is required.

13. Screenshots / Demonstration

```
"C:\Users\Anin shaj\PycharmProjects\PythonProject\.venv\Scripts\python.exe" "C:\Users\Anin shaj\PycharmProjects\PythonProject\miniproject.py"

=====
      BANK MANAGEMENT SYSTEM
=====
1. Register
2. Login
3. Exit

Enter your choice: 1

=====
      USER REGISTRATION
=====
Enter username: aninshaj
Enter email: aninshaj@gmail.com
Enter password: Anin@1234

Username or email already exists!
```

Fig. 1 — User registration

```
=====
                        LOGIN
=====
Enter username: anjopaul
Enter password: anjo@1234

Login successful!
Welcome, anjopaul

=====
                        CUSTOMER DASHBOARD
=====
1. Create Bank Account
2. View Account Details
3. Check Balance
4. Deposit Money
5. Withdraw Money
6. Transfer Money
7. Transaction History
8. View Profile
9. Update Profile
10. Logout
```

Fig. 2 — Login and customer dashboard

DB Browser for SQLite - C:\Users\Anin shaj\PycharmProjects\PythonProject\bankmanagement.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo Open Project Save Project Attach Database Close Database

Database Structure Browse Data Edit Pragmas Execute SQL

Table: account

	account_no	user_id	name	age	account_type	balance
	Filter	Filter	Filter	Filter	Filter	Filter
1	100001	2	Anjo Paul	21	Savings	171000.0
2	100002	1	Anin shaj	21	Current	2000056.0

Fig. 3 — Account table in DB Browser

```
=====
                        DEPOSIT MONEY
=====
Enter amount to deposit: ₹200000

Deposit successful!
Deposited Amount : ₹200000.00
New Balance      : ₹200000.00

=====
                        CUSTOMER DASHBOARD
=====
```

Fig. 4 — Deposit operation

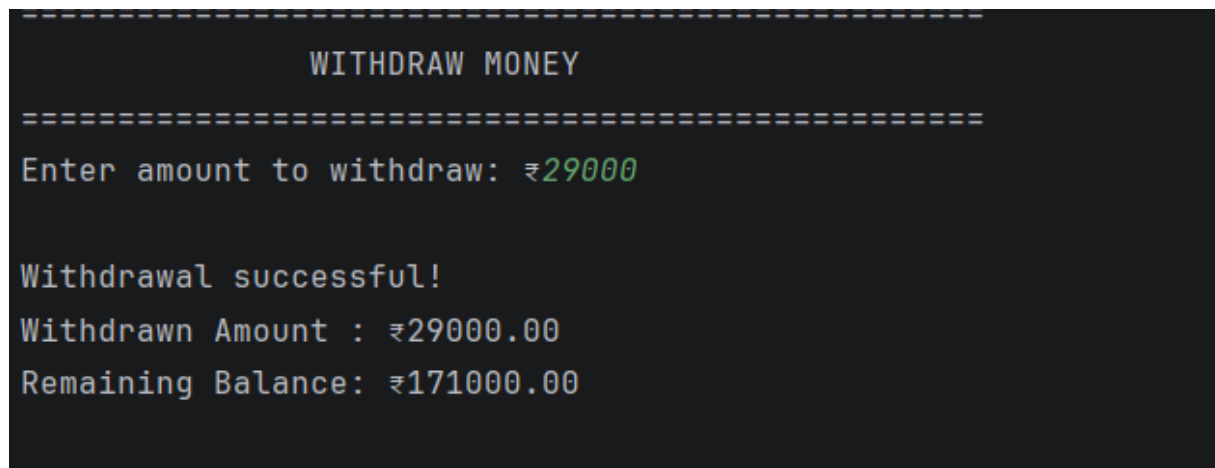


Fig. 5 — Withdrawal operation

DB Browser for SQLite - C:\Users\Anin shaj\PycharmProjects\PythonProject\bankmanagement.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo Open Project Save Project Attach Database Close Database

Database Structure Browse Data Edit Pragma Execute SQL

Table: transactions

	transaction_id	account_no	transaction_type	amount	description	date_time
	Filter	Filter	Filter	Filter	Filter	Filter
1	1	100001	DEPOSIT	200000.0	Money deposited	2026-09-06 12:17:22
2	2	100001	WITHDRAW	29000.0	Money withdrawn	2026-09-06 12:17:32
3	3	100002	DEPOSIT	2000056.0	Money deposited	2026-09-06 12:18:41

Fig. 6 — Transaction history in DB Browser

DB Browser for SQLite - C:\Users\Anin shaj\PycharmProjects\PythonProject\bankmanagement.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo Open Project Save Project Attach Database Close Database

Database Structure Browse Data Edit Pragma Execute SQL

Table: sqlite_sequence

	name	seq
	Filter	Filter
1	registration	2
2	transactions	3

Fig. 7 — Database structure showing the three tables

user_id	username	email	password
1	aninshaj	aninshaj@gmail.com	affa77d26fe70b615d6b77ec3d5159d6a6b1...
2	anjopaul	anjopaul@gmail.com	a6006ea9b35730bce6aa3ef64803336f5aee...

Fig. 8 — Registration data in DB Browser

14. Testing

Test case	Expected result	Observed result
Register new user	User record is created	Successful
Login with valid credentials	Dashboard is displayed	Successful
Create bank account	Account number and balance are created	Successful
Deposit money	Balance increases and transaction is stored	Successful
Withdraw money	Balance decreases when sufficient funds exist	Successful
Transfer to invalid account	Transfer is rejected	Rejected as expected
View transaction history	Stored transactions are displayed	Successful

15. Results

The completed application successfully demonstrates the core workflow of a small banking system. User information, account information and transaction records are stored in SQLite, while the console interface provides the main banking operations.

16. Advantages

- Simple and easy-to-use menu-driven interface.
- Local database storage without external database-server setup.
- Clear separation of user, account and transaction data.
- Transaction records provide traceability.
- Password hashing improves over plain-text password storage.

17. Limitations

- The application is console-based rather than graphical.
- It is intended for educational use and not real banking deployment.

- SQLite is suitable for this small project but is not a replacement for a production banking database architecture.
- The security model is basic and can be strengthened further for production use.

18. Future Scope

- Graphical user interface or web interface.
- Admin dashboard and role-based access.
- Loan and interest management.
- Password reset and stronger password-storage methods.
- Improved audit logging and security controls.
- Reports and account statements.

19. Conclusion

The Bank Management System provides a practical demonstration of Python programming, SQLite database management, relational data modeling and basic banking operations. The project integrates authentication, account management and transaction processing into one working application and provides a foundation for future expansion into a more advanced banking system.

20. References

- Python Standard Library documentation — sqlite3, hashlib, re and datetime modules.
- SQLite documentation and DB Browser for SQLite for database inspection.
- PyCharm IDE for development and testing.